Instituto Superior de Formación Técnica Nº 151
Carrera: Analista en Sistemas
3 Año. Algoritmos y Estructuras de Datos III.
Alumna LUZURIAGA Silvana

GRAFOS.

Marco Teórico:

1. Un grafo G es una estructura matemática que agrupa entes físicos o conceptuales y las relaciones entre ellos. Esto formado por un conjunto de vértices o nodos (V) y un conjunto de arcos o aristas (A). Se representa con el par G=(V, A)

2. El concepto de Vértice y Arco:
   * Vértice (o nodo): Representa a los entes u objetos dentro de nuestro grafo.
   * Arco (o arista): Representa la relación entre dos nodos. Al estar formada por dos nodos, se representa tipicamente por el **par (u, v)**. Si el grafo es dirigido, los arcos tienen una dirección especifica indicada por una flecha.

3. Para reemplazar un grafo en la memoria del ordenador se utilizan principalmente dos implementaciones: la **Matriz de Adyacencia** y las **Listas de Adyacencia.**
   * **Matriz de Adyacencia**: Se utiliza un array bidimensional. Es ideal y mas útil para grafos densos (que tienen la mayoría de los arcos posibles), ya que el acceso a sus elementos es rápido.
   * **Listas de adyacencia**: Es una estructura dinámica multienlazada (una tabla directorio donde cada elemento representa un vértice que apunta a una lista de sus adyacentes). Son mucho mas eficientes y útiles para grafos dispersos (aquellos con relativamente pocos arcos), ya que evitan el desperdicio de memoria que generarían los múltiples ceros en una matriz.

4. Recorrer un grafo consiste en visitar todos los vértices alcanzables a partir de uno dado, manteniendo unn registro de los nodos ya marcados. Existen **dos formas de hacerlo**:
   *** Recorrido en Anchura (usa una Cola):**
   1. Marcar el vértice de partida ***v***.
   2. Meter en la cola el vértice de partida ***v***.
   3. Repetir los ***pasos 4 y 5*** hasta que se cumpla la condición de cola vacía.
   4. Quitar el nodo frente de la cola, ***w***, y visitar ***w***.
   5. Meter en la cola todos los vértices adyacentes a ***w*** que no estén marcados, t marcarlos a continuación.
   6. Fin del recorrido.

   *** Recorrido en Profundidad (usa una Pila)**:
   1. Empezar por un vértice ***v***, marcarlo como visitado y meterlo en la pila.
   2. Iterativamente quitar el nodo cabeza de la pila y procesarlo.
   3. Recorrer en profundidad cada vértice adyacente a ***v*** no visitado (es decir, la dirección de visitar es hacia adelante mientras sea posible).

4. Meter en la pila los adyacentes no marcados y marcados.
5. Repetir hasta que no haya mas vértices adyacentes no visitados.

5. Un grafo (no dirigido) es conexo si existe un camino entre cualquier par de nodos (vértices) que forman el grafo. Si el grafo es dirigido y cumple esta misma propiedad, se dice que es **fuertemente conexo.**

6. Dado un grafo ***G*** con ***n*** vértices, la matriz de caminos(también llamada ***P***) es una ***matriz de n x m elementos*** donde un elemento P_{ij} ***= 1 si hay un camino*** desde el vértice v_i al vértice v_j, y **vale 0 si no existe tal camino.**

7. El cierre transitivo (o contorno transitivo) de un grafo ***G*** es otro grafo ***G´***que posee los mismos vértices y cuya matriz de adyacencia es la matriz de caminos de G. Muestra todas las conectividades indirectas. Ejemplo: imagina un grafo donde existe un arco del vértice A al vértice B, y otro del vértice B al vértice E. El cierre transitivo revelará que existe un camino de longitud 2 que va desde A hasta E, marcándolo en la matriz con un 1.

8. El ordenamiento topológico se aplica estrictamente sobre grafos dirigidos sin ciclos (**GDA- Grafos Dirigido Aciclico**). Se aplica de forma ideal para modelar las relaciones que existen entre las diferentes tareas que deben terminarse para finalizar un proyecto. Por ejemplo, en una estructura de pre-requisitos de cursos académicos, donde un curso debe completarse obligatoriamente antes de empezar otro.

9. Se utiliza para calcular la matriz de caminos (cierre transitivo) de un grafo representado por su matriz de adyacencia. Su uso permite saber rápidamente si existe conectividad (un camino simple de cualquier longitud) entre cualquier par de vértices del grafo.

10. El algoritmo de Dijkstra se utiliza para encontrar el camino de longitud o coste mínimo desde un único vértice origen al resto de vértices en grafos dirigidos y valorados con factores de peso positivos.
    *Ejemplo: Encontrar la mejor ruta o el trayecto que requiera el menor tiempo posible para ir desde un lugar inicial hacia un conjunto de centros dentro de una ciudad.

11. Es un subconjunto de un grafo no dirigido conectado(una red) que abarca a todos los vértices del grafo y los conecta sin formar ningún ciclo.

12. Ambos algoritmos buscan el árbol de expansión de coste mínimo, pero difiere en su estrategia voraz:
    *** Algoritmo de Prim**: comienza desde un vértice inicial arbitrario y hace crecer un único árbol de forma continúa. En cada iteración, se busca y agrega el vértice no incluido cuya arista de conexión con el árbol actual sea la de menor coste.
    *** Algoritmo de Kruskal**: empieza con cada vértice del grafo como un arbol separado(formando un bosque). evalúa las aristas del grafo ordenadas de menor a mayor peso y en cada paso conecta dos árboles diferentes en uno solo usando el arco de menor longitud. Rechaza cualquier arista que forme un ciclo.

13. Los grafos en memoria se implementan como tipos abstractos de datos para resolver una multitud de problemas reales. Sus utilidades principales abarcan:
    * Sociología, química, geografía e ingeniería eléctrica e industrial.
    * Planificación de las tareas necesarias para completar un proyecto.
    * Encontrar las rutas de menor longitud entre dos puntos geográficos.
    * Cálculo de flujos máximos desde una fuente.

Marco Practico:

Para implementar la solución que nos pide un “camino de menor coste entre los nodos” de modo que nuestro Agente de Ventas pueda recorrer o conectar todo su itinerario, el enfoque adecuado es encontrar el **Arbol de Expansion de Coste Minimo** de todo el grafo. Emplearemos el **Algoritmo de Kruskal**.

PASO 1. IDENTIFICACION Y ORDENAMIENTO DE ARISTAS(POR PESO CRECIENTE)

Analizando el esquema, los costes de las aristas son:

1. F - E = 1.
2. A - D =2.
3. A - B = 3.
4. B - E = 4.
5. A - C = 5.
6. B - C = 5.
7. H - B = 6.
8. B - G = 6 .
9. C - F = 7.
10. D - B = 8 .
11. H - F = 9.
12. C - G = 9.
13. A - H = 10.
14. D -G = 12.
15. H - D = 14.
16. E - G = 15

PASO 2. EJECUCIÓN DEL ALGORITMO DE KRUSKAL

Comenzamos con un bosque donde cada nodo (A, B, C, D, E, F, G, H) esta separado. Vamos agregando las aristas de menor peso siempre que no formen ciclos:

* Agregamos F - E (Peso 1).
* Agregamos A - D(Peso 2).
* Agregamos A - B(Peso 3). (Conecta el bloque de A-D con B).
* Agregamos B - E(Peso 4). (Conecta el bloque A-D-B con el bloque F-E).
* Agregamos A - C(Peso 5).
* Agregamos B - C(Peso 5): Se descarta porque cerraría un ciclo (A-B-C).
* Agregamos H - B(Peso 6). (Conecta a H con la red principal).
* Agregamos B - G(Peso 6).(Conecta a G con la red principal):

Resultado Final (Ruta de menor coste)
la red ya tiene conectados los 8 nodos usando exactamente 7 aristas. Las aristas que forman el Árbol de Expansión Mínima son: **{F-E, A-D, A-B, B-E, A-C, H-B, B-G).**
El coste total del itinerario óptimo para conectar todas las ciudades es la suma de estos pesos: 1+2+3+4+5+6+6 = 27